

Multi-Church Deploy Guide

Church CMS — Push updates to Zion Hub & all client churches

Easy reference for Push-All deploy helper

This guide explains how to publish website updates to every church from one place — without repeating long git push and Vercel steps each time.

1. The simple idea

You develop the CMS once. Each church has its own website (Vercel) and database (Supabase). When you change the UI, you need to send the same code to every church's GitHub repo, and sometimes force a Vercel production build.

Church	Git remote	Live website
Zion Hub	origin	cms-three-mu.vercel.app
St Pauls	stpauls	cms-trichystpaulschurch.vercel.app

Why not only one push?

Today each church can use its own GitHub repo. Pushing to origin updates Zion Hub only. St Pauls needs a second push (remote: stpauls). On Hobby Vercel plans, Cursor co-authored commits can also block auto-deploy — so the helper runs Vercel CLI production deploy for St Pauls.

2. What we built for you

A small helper lives in the deploy folder:

- deploy\Push-All.bat — double-click menu (easiest)
- deploy\Push-All.ps1 — PowerShell engine
- deploy\clients.json — list of churches (add more later)

One run can:

1. Commit your changes with a clean message (no Cursor co-author trailer)
2. Push to every git remote in clients.json
3. Run Vercel production deploy for churches that need it (St Pauls)

3. How to use (everyday)

Option A — Double-click (recommended)

1. Open the project folder: Church-CMS-React
2. Go into the deploy folder
3. Double-click Push-All.bat
4. Choose an option:

- 1) Push all remotes + Vercel prod
- 2) Push all remotes only (skip Vercel CLI)
- 3) Dry run (show what would happen)
- 4) St Pauls only (push + Vercel)
- 5) Exit

If you have uncommitted files, the script asks for a commit message. Type a short message and press Enter. Leave blank to skip committing and only push what is already committed.

Option B — From terminal

```
cd C:\Projects\Church-CMS-React
npm run deploy:all
npm run deploy:stpauls
npm run deploy:dry
```

Option C — PowerShell with flags

```
powershell -ExecutionPolicy Bypass -File .\deploy\Push-All.ps1
powershell -ExecutionPolicy Bypass -File .\deploy\Push-All.ps1 -Message "Fix receipts print"
powershell -ExecutionPolicy Bypass -File .\deploy\Push-All.ps1 -SkipVercel
powershell -ExecutionPolicy Bypass -File .\deploy\Push-All.ps1 -Only stpauls
powershell -ExecutionPolicy Bypass -File .\deploy\Push-All.ps1 -DryRun
```

4. What “success” looks like

- Each remote shows: OK Pushed to .../main
- St Pauls Vercel shows: OK Live: <https://cms-trichystpaulschurch.vercel.app>
- Open the live site !' hard refresh (Ctrl+F5) !' confirm your change

First time Vercel CLI

If asked to log in, use the church Vercel account that owns that project (for St Pauls: trichystpaulschurch). Stay logged in so future runs are quiet.

5. Adding a new church later

1. Create the church GitHub repo and Vercel project (with that church's Supabase env vars).
2. Add a git remote on your PC:

```
git remote add stmarys git@github.com:ORG/StMarys-CMS.git
```

3. Edit deploy\clients.json and add a block like St Pauls:

```
{
  "id": "stmarys",
  "label": "St Marys",
  "remote": "stmarys",
  "siteUrl": "https://cms-stmarys.vercel.app",
  "vercelDeploy": true,
  "vercelProject": "cms-stmarys",
  "vercelOrgId": "team_...",
  "vercelProjectId": "prj_..."
}
```

4. Get Org ID / Project ID from the church's `.vercel/project.json` after linking once, or from the Vercel dashboard.

5. Run Push-All again — the new church is included automatically.

Set `vercelDeploy` to `false` if GitHub! Vercel auto-deploy works reliably for the commit authors match the Vercel owner).

6. Troubleshooting

I pushed but the live site has no change

- Hard refresh the browser (Ctrl+F5) or try a private window.
- Open Vercel! Deployments. If status is BLOCKED, Hobby blocked the Git
- Fix: run menu option 1 or 4 so the script does `npx vercel --prod`.

Error: commit author does not have contributing access

Hobby teams do not allow collaboration. Commits co-authored by Cursor (or authored by someone who is not the Vercel owner) get blocked.

- Push-All creates clean commits and CLI-deploys St Pauls — use it instead of manual git push only.
- Long-term options: upgrade that Vercel team to Pro, or always deploy via this script.

Remote missing

```
git remote -v
git remote add stpauls git@github.com:trichystpaulschurch-del/StPauls-CMS.git
```

Wrong church / skipped Vercel

- Check `deploy\clients.json` — `vercelDeploy` must be true for CLI deploy.
- Confirm you are logged into the correct Vercel account: `npx vercel whoami`

7. What this helper does NOT deploy

Frontend (React UI) is covered. These still need a separate per-church step:

- Supabase SQL migrations / new tables
- Edge Functions (`cms-full-backup`, `cms-provision`, ...)
- Supabase secrets (Google OAuth, payment keys, ...)
- Google Drive backup folder setup

See also: `docs\MULTI_CHURCH_DEPLOYMENT_GUIDE.md` for the full multi-church architecture.

8. Quick cheat sheet

Goal	Do this
Update all churches	Double-click Push-All.bat ! option 1
Preview only	Option 3 (Dry run)
St Pauls only	Option 4
Git push only	Option 2
Add new church	git remote add + edit clients.json

Remember

After any successful deploy, always verify on the live church URL — not only on localhost.

Generated for Church CMS · deploy/Push-All · Keep this PDF with the project for future reference.